Cours Modélisation et vérification des systèmes informatiques
Exercices
Modélisation TLA$^+$ (1)
par Dominique Méry
16 octobre 2026

Exercice 1 *appex1_1.tla, appex1_1.cfg*
L'accès à une salle est contrôlé par un système permettant d'observer les personnes qui entrent ou qui sortent de cette salle. Ce système est un ensemble de capteurs permettant d'identifier le passage d'une personne de l'extérieur vers l'intérieur et de l'intérieur à l'extérieur. Le système doit garantir qu'au plus`max` *personnes soient dans la salle.*
Ecrire un module TLA$^+$ permettant de modéliser un tel système respectant la propriété attendue.

◇– **Solution de l'exercice 1**
Nous donnons trois solutions possibles selon notre analyse :

— la première solution propose deux actions *entrer* et *sortir* et on définit une relation de transition *next*. On définit un modèle en instanciant *max* et en testant les deux propriétés de sûreté *question1* et *safety1*. Les deux questions produisent un échec et donc la propriété de sûreté n'est pas vérifiée.
— la deuxième solution propose deux actions *entrer2* et *sortir* et on définit une relation de transition *next2*. On définit un modèle en instanciant *max* et en testant les deux propriétés de sûreté *question1* et *safety1*. Il n'y a pas de retour sur la question *safety1*.
— la troisième solution propose deux actions *entrer2* et *sortir2* et on définit une relation de transition *next3*. On définit un modèle en instanciant *max* et en testant les deux propriétés de sûreté *question1* et *safety1*. L'utilisation de l'outil conduit à la vérification de la propriété de *safety1*.

Listing 1 – Module TLA+

```
1  ------------------------------- MODULE appex1_1 ---------------------
2  EXTENDS Naturals, Integers, TLC
3  ------------------------------------------------------------------------
4  CONSTANTS max
5  ------------------------------------------------------------------------
6  VARIABLES  np
7  ------------------------------------
8  (*  tentative 1 *)
9  entrer ==
10     /\ TRUE
11     /\ np'=np +1
12 sortir == np'=np-1
13 next1 == entrer \/ sortir
14 Init == np=0
15 ------------------------------------------------------------------------
16 (* tentative 2 *)
17 entrer2 == np<max /\ np'=np+1
18 next2 ==  entrer2  \/  sortir
19 ------------------------------------------------------------------------
20 (* tentative 3 *)
21 sortir2 == np>0 /\ np'=np-1
22 next3 ==  entrer2  \/  sortir2
23 ------------------
24 Safety1 == np \in Int
25 Safety2 == np \leq max
26 Safety3 == np \geq 0
27 qq ==  np # 89
28
29 Safety ==  Safety1 /\ Safety2 /\ Safety3
30 Test == Safety
```

```
31
32 ==========================================================================
```

Listing 2 – Configuration TLC

```
1  \* SPECIFICATION
2  \* Uncomment the previous line and provide the specification name if it's
       declared
3  \* in the specification file. Comment INIT / NEXT parameters if you use
       SPECIFICATION.
4
5  CONSTANTS
6      greeting = "Hello"
7      max = 19
8      minint = 1000
9      maxint =  1000
10
11 INIT Init
12 NEXT next3
13
14
15 \* PROPERTY
16 \* Uncomment the previous line and add property names
17
18  INVARIANT
19 \* Uncomment the previous line and add invariant names
20 \* SPECIFICATION
21 \*    Safety2
22  \*   Safety1
23  \*   Safety3
24   \*  qq
25     Test
```

Fin 1

Exercice 2 *Le PGCD de deux nombres vérifie les propriétés suivantes :*

- $\forall a, b \in \mathbb{N}.pgcd(a, b) = pgcd(b, a)$
- $\forall a, b \in \mathbb{N}.pgcd(a, a+b) = pgcd(a, b)$
- *Ecrire une spécification TLA$^+$ calculant le PGCD de deux nombres donnés.*
- *Donner une explication ou une justification de la correction de cette solution*

◇– **Solution de l'exercice 2**

Listing 3 – Module TLA+

```
1  ------------------ MODULE appex1_2 -----------------
2  (* Computing  GCD of two numbers a and b *)
3  EXTENDS Naturals,TLC
4  CONSTANTS a,b
5  VARIABLES  x,y
6  ------------------------------------
7  ASSUME a \in Nat /\ b \in Nat
8  Init == x=a /\ y=b
9  ------------------------------------
10 a1 ==
11     /\ x > y
12     /\ x'=x-y
13     /\ y'=y
14 a2 ==
15     /\ x < y
16     /\ y'=y-x
```

```
    /\ x'=x
over == x=y /\ x'=x /\ y'=y
------------------------------------
Next ==
    \/ a1
    \/ a2
    \/ over

------------------------------------
(* Safety properties to be verified *)
question == x # y
prop1 == [](x \geq 0 /\ y \geq 0)
prop2 == x+y \leq a+b
prop == question
Check == prop2
=============================================
```

Listing 4 – Configuration TLC

```
\* SPECIFICATION
\* Uncomment the previous line and provide the specification name if it's
    declared
\* in the specification file. Comment INIT / NEXT parameters if you use
    SPECIFICATION.

CONSTANTS
    greeting = "Hello"
    a = 99
    b = 56

INIT Init
NEXT Next

PROPERTY
\* Uncomment the previous line and add property names
    prop1
INVARIANT
\* Uncomment the previous line and add invariant names
Check

CHECK_DEADLOCK FALSE
```

Fin 2

Exercice 3 *appex1_3.tla,appex1_3.cfg, petri10*

Un réseau de Petri est un uple $R = (S, T, F, K, M, W)$ *avec*

- S *est l'ensemble (fini) des places.*
- T *est l'ensemble (fini) des transitions.*
- $S \cap T = \varnothing$
- F *est la relation du flôt d'exécution :* $F \subseteq S \times T \cup T \times S$
- K *représente la capacité de chaque place :* $K \in S \nrightarrow Nat$.

La fonction K affecte à toute place s de S, le nombre maximum de jetons que celle-ci puise contenir. Si cette fonction n'est pas définie, alors la capacité est infinie.

- M *représente le initial marquage chaque place :*
  $M \in S \rightarrow Nat$ *et vérifie la condition* $\forall s \in S : M(s) \leq K(s)$.
- W *représente le poids de chaque arc :* $W \in F \rightarrow Nat$
- *un marquage M pour R est une fonction de S dans Nat :*
  $M \in S \rightarrow Nat$ *et respectant la condition* $\forall s \in dom(K) : M(s) \leq K(s)$.
- *une transition t de T est activable à partir de M un marquage de R si*

1. $\forall s \in \{ s' \in S \mid (s',t) \in F \} : M(s) \geq W(s,t)$.
2. $\forall s \in \{ s' \in S \mid (t,s') \in F \} : M(s) \leq K(s) - W(s,t)$.

— *Pour chaque transition t de T, Pre(t) est l'ensemble des places conduisant à t et Post(t) est l'ensemble des places pointées par un lien depuis t :*
Pre(t) = { s' ∈ S : (s',t) ∈ F } et Post(t) = { s' ∈ S : (t,s') ∈ F }

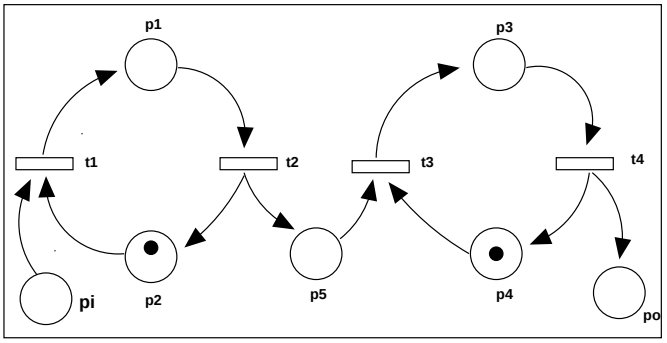
— *Soit une transition t de T activable à partir de M un marquage de R :*

1. $\forall s \in \{ s' \in S \mid (s',t) \in F \} : M(s) \geq W(s,t)$.
2. $\forall s \in \{ s' \in S \mid (t,s') \in F \} : M(s) \leq K(s) - W(s,t)$.

— *un nouveau marquage M' est défini à partir de M par : '* $\forall s \in S$,

$$M'(s) = \begin{cases} M(S) - W(S,T), \text{ SI } S \in \text{PRE}(T) - \text{POST}(T) \\ M(S) + W(T,S), \text{ SI } S \in \text{POST}(T) - \text{PRE}(T) \\ M(S) - W(S,T) + W(T,S), \text{ SI } S \in \text{PRE}(T) \cap \text{POST}(T) \\ M(S), \text{ SINON} \end{cases}$$

On considère le réseau suivant :



Question 3.1 *Traduire ce réseau en un module TLA$^+$ dont le squelette est donné dans le texte. Pour cela, on donnera la définition des quatre transitions $t1$, $t2$, $t3$,$t4$. On ne tiendra pas compte de la capacité des places : les places ont une capacité d'au plus un jeton, sauf la place pi qui peut contenir N jetons, la place p5 peut contenir au plus B jetons et la place po peut contenir au plus Q.*

Question 3.2 *Donner une relation liant les places po,p1,p3,p5,pi et la valeur N. Justifiez votre réponse.*

Question 3.3 *Si on suppose que la place po peut contenir au plus Q jetons, donnez une condition sur Q pour que tous les jetons de pi soient consommés un jour. Justifiez votre réponse.*

Question 3.4 *Expliquez ce que modélise ce réseau de Petri.*

◇– Solution de l'exercice 3

Listing 5 – Module TLA+

```
1  ------------------------------- MODULE appex1_3
       ---------------------------------
2  (* petri10 *)
3  EXTENDS Naturals,TLC
4  CONSTANTS Places,N,Q,B
5  VARIABLES  M
6
7  ----------------------------
8
9  t1 ==
10         /\ M["p2"]  \geq 1  /\  M["pi"]  \geq 1
11         /\ M'=  [[[M EXCEPT!["p1"]= 1]
```

```
                      EXCEPT![\"pi\"]= @-1]
                      EXCEPT![\"p2\"]=@-1]

t2 ==
        /\ M[\"p1\"] \geq 1 /\ M[\"p5\"] <  B
        /\ M'=  [[[M EXCEPT![\"p1\"]=@-1]
                      EXCEPT![\"p5\"]=M[\"p5\"]+1]
                      EXCEPT![\"p2\"]=1]

t3 ==
        /\ M[\"p5\"] \geq 1 /\ M[\"p4\"] \geq 1
        /\ M'=  [[[M EXCEPT![\"p3\"]=@+1]
                      EXCEPT![\"p5\"]=@-1]
                      EXCEPT![\"p4\"]=@-1]



t4 ==
        /\ M[\"p3\"] \geq 1 /\ M[\"po\"]<Q
        /\ M'=  [[[M EXCEPT![\"p3\"]= M[\"p3\"]-1]
                 EXCEPT![\"po\"]=M[\"po\"]+1]
                 EXCEPT![\"p4\"]=M[\"p4\"]+1]

-----------------------------

Init1 ==  M = [p \in Places |-> IF p \in {\"p4\",\"p2\"} THEN 1 ELSE
                IF p = \"pi\" THEN N ELSE 0 ]
Init == Init1
Next == t1 \/ t2 \/ t3 \/ t4 \/ M'=M


-----------------------------

TypeInvariant ==  \A p \in Places :  M[p] \geq 0

Inv1 ==  M[\"pi\"]+M[\"p5\"]+M[\"po\"]+M[\"p1\"]+ M[\"p3\"]  = N

Inv2 ==  M[\"po\"] \leq  Q

QInv4 ==  M[\"pi\"]+M[\"p5\"]+M[\"po\"]+M[\"p2\"]+M[\"p4\"] = N+2
Inv5 ==  M[\"p3\"]+M[\"p4\"]+M[\"p1\"]+M[\"p2\"] = 2

Inv3 ==  M[\"p3\"] = 0

Inv6 == M[\"p1\"] + M[\"p2\"] = 1

Question == M[\"po\"] #  N
Safety1 ==  M[\"p2\"] \leq  1  /\ M[\"p2\"] \geq  0
Check == Inv6
safe == Inv3
=============================================
```

Listing 6 – Configuration TLC

```
\* SPECIFICATION
\* Uncomment the previous line and provide the specification name if it's
    declared
\* in the specification file. Comment INIT / NEXT parameters if you use
    SPECIFICATION.

CONSTANTS
```

```
    greeting = "Hello"
    B = 4
    N = 9
    Q = 10
    Places = {"pi","po","p1","p2","p3","p4","p5"}

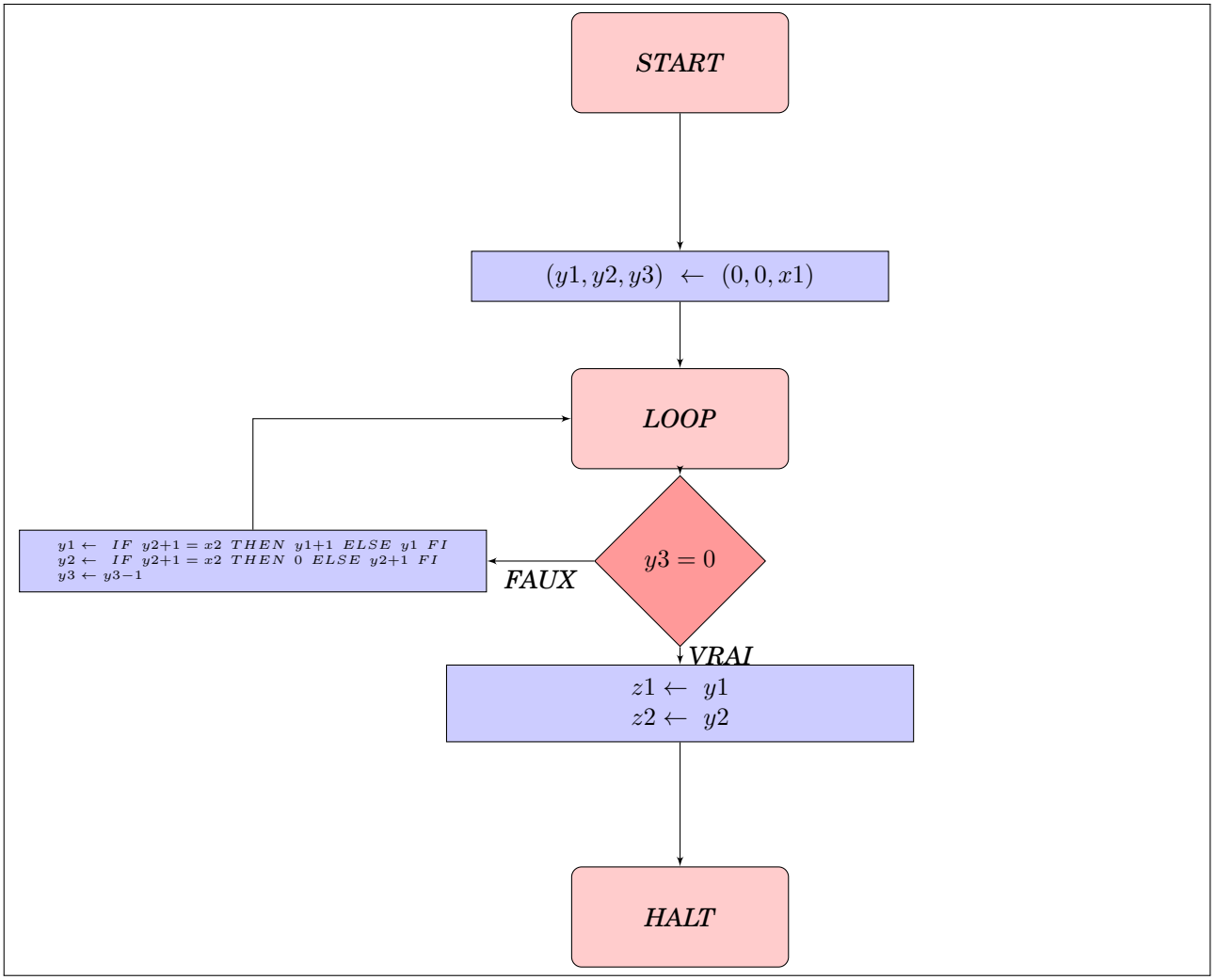
INIT Init
NEXT Next

\* PROPERTY
\* Uncomment the previous line and add property names

INVARIANT
\* Uncomment the previous line and add invariant names
    safe
```

Fin 3

Exercice 4 *appex1_4*
On considère l'algorithme suivant décrit par un organigramme ou flowchart :



Question 4.1 *Traduire cet algorithme sous forme d'un module TLA$^+$.*

Question 4.2 *Ecrire un contrat pour cet algorithme.*

Question 4.3 *Tester les valeurs des variables à l'exécution.*

Question 4.4 *Montrer que cet algorithme est partiellement correct par rapport à sa précondition et à sa postcondition.*

◇– **Solution de l'exercice 4**

Listing 7 – Module TLA+

```
---------------------------------- MODULE appex1_4
    --------------------------------
(* modules de base importables *)
EXTENDS Naturals, Integers, TLC
-----------------------------------------------------------------------------
CONSTANTS  x10,x20,U,MAX

MIN == -MAX
-----------------------------------------------------------------------------
VARIABLES  x1,x2,y1,y2,y3,z1,z2,pc
-----------------------------------------------------------------------------
locs == {"START","HALT","LOOP"}
-----------------------------------------------------------------------------
BF(X) == X#U => X \in MIN..MAX

ASSUME  BF(x10) /\ BF(x20)
-----------------------------------------------------------------------------


Init == pc="START" /\ x1=x10 /\ x2=x20 /\ y1=U /\ y2 = U /\ y3=U /\ z1 = U /\
    z2 = U
-----------------------------------------------------------------------------
actionSTART_LOOP ==
               /\ pc = "START"
               /\ pc'="LOOP"
               /\ y1'=0
               /\ y2'=0
               /\ y3'=x1
               /\ UNCHANGED <<x1,x2,z1,z2>>


actionLOOP_HALT ==
              /\ pc="LOOP"
              /\ y3=0
              /\ pc'="HALT"
               /\ y1'=y1
               /\ y2'=y2
               /\ y3'=y3
               /\ z1'=y1
               /\ z2'=y2
               /\ UNCHANGED <<x1,x2>>



actionLOOP_LOOP ==
              /\ pc="LOOP"
              /\ y3#0
              /\ pc'=pc
              /\ y1'= IF y2+1=x2 THEN y1+1 ELSE y1
              /\ y2'= IF y2+1 = x2 THEN 0 ELSE y2+1
              /\ y3'=y3-1
              /\ z1'=z1
              /\ z2'=z2
```

```
52                /\ UNCHANGED <<x1,x2>>
53
54
55  skip == UNCHANGED <<x1,x2, y1,y2,y3,z1,z2,pc>>
56
57  ----------------------------------------------------------------------------
58
59
60  Next ==
61      \/ actionSTART_LOOP
62      \/  actionLOOP_HALT
63      \/ actionLOOP_LOOP
64      \/ skip
65
66
67  ----------------------------------------------------------------------------
68
69  (* vÃ©rification du contrÃ´le *)
70  safety1 == pc \in locs
71  (* correction partielle *)
72  safety2 == pc="HALT" => z1 = x1 \div x2 /\ z2= x1 % x2 /\ PrintT(z1) /\ PrintT(
        z2)
73  safety3 == pc="HALT" =>      x1 = z1*x2 +z2  /\ 0 \leq z2 /\ z2 < x2
74  (* vÃ©rification de l'absence d'erreurs Ã  l'exÃ©cution ou RTE *)
75  safety4 == BF(z1) /\BF(z2) /\BF(y1) /\BF(y2) /\BF(y3)
76
77  prop1 == [] (x1=x10 /\ x2=x20)
78
79
80  Safety == safety1 /\ safety2  /\ safety3  /\ safety4
81  ============================================================================
```

Listing 8 – Configuration TLC

```
1   \* SPECIFICATION
2   \* Uncomment the previous line and provide the specification name if it's
        declared
3   \* in the specification file. Comment INIT / NEXT parameters if you use
        SPECIFICATION.
4
5   CONSTANTS
6       x10 = 46
7       x20 = 45
8
9       MAX = 1000
10      U = 1001
11
12  INIT Init
13  NEXT Next
14
15  \* PROPERTY
16  \* Uncomment the previous line and add property names
17
18  INVARIANT
19  \* Uncomment the previous line and add invariant names
20  Safety
```

Fin 4